

Claude как библиотека.

Опциональный этап для тех, кто хочет идти дальше. До этого мы использовали Claude через CLI и через Claude API внутри backend (этап 06). Здесь – **Claude Agent SDK**: тот же агент с циклом «prompt → tools → result», но запускаемый из ваших Python/Node-скриптов. Без Claude Code-обёртки. Headless. Готов работать в Twitter-демоне, Telegram-боте, GitHub Actions.

ДЛИТЕЛЬНОСТЬ

7–10 дней • 2–4 ч/день

ЧТО ВЫ ОСВОИТЕ

Agent SDK • agent loop • headless • CI/CD • daemon

ГЛАВНЫЙ ИТОГ

Claude встроен в ваши скрипты и инфраструктуру

АРТЕФАКТ

Twitter-демон + AI-консьерж в Telegram + автоматический code review в PR

⚠ Этот этап – для «следующего шага». До этапа 10 включительно курс был «AI-нативный без программирования». Этап 11 – уже про *программирование с Claude*: вы будете писать (точнее, направлять Claude писать) Python-скрипты, которые сами вызывают Claude API. Если этого не нужно – пропускайте, переходите к капстоуну.

ОБЯЗАТЕЛЬНО

ANTHROPIC_API_KEY + skill claude-api

Что это: ключ от Anthropic API (этап 06) + skill, который учит Claude писать код с Anthropic SDK правильно: prompt caching, выбор модели, обработка ошибок.

Зачем сейчас: весь этап стоит на них. Без ключа SDK не запустится; без skill код будет «плоским» и дорогим.

Где взять: ключ на console.anthropic.com; skill `claude-api` – *встроенный*, ставить ничего не надо (см. этап 06). Проверка – `/skills`.

ОБЯЗАТЕЛЬНО • СРЕДА

Python 3.10+ или Node.js 20+

Что это: язык, на котором мы будем запускать Claude как библиотеку. В курсе – Python, потому что у нас уже есть python-окружение из этапа 03. Если предпочитаете Node – Anthropic SDK для него тоже есть.

Зачем сейчас: Claude Agent SDK поставляется как пакет под конкретный язык.

Установка пакета:

```
# Python
$ pip install anthropic

# Node.js
$ npm install @anthropic-ai/sdk
```

ЖЕЛАТЕЛЬНО • АККАУНТ

X (Twitter) Developer + ключи API

Что это: доступ к X API для первого проекта (Twitter-сигналы).

Зачем сейчас: чтобы стрим твитов попадал в ваш демон.

Где взять: developer.x.com. Бесплатный тариф ограничен; для учебного проекта

может хватить либо «basic» публик стрим. Если не хочется регистрироваться — в проекте есть «офлайн-демо» из готового файла твитов.

Куда положить: `X_BEARER_TOKEN` в `.env`.

ОБЯЗАТЕЛЬНО

Из этапов 06 и 10

- Telegram-бот из этапа 10 (для push-уведомлений и консьержа);
- Backend на Railway (этап 06) — будем туда деплоить демон.

Claude Code, Claude API и Claude Agent SDK — в чём разница

Три способа использовать Claude:

- 1. **Claude Code (CLI)** – интерактивный диалог в терминале. Это всё, что мы делали с этапа 00 по 10. Удобно для *работы человека с Claude*.
- 2. **Claude API (chat completions)** – HTTP-вызов «вход → текст». Этап 06: ваш backend шлёт промпт, получает ответ. Удобно для *встраивания одного ответа* внутрь приложения. Без tools, без цикла.
- 3. **Claude Agent SDK** – полный «агентский» цикл из вашего скрипта: *prompt → tool calls → tool results → next prompt → ...* до завершения. С agent loop, tools, prompt caching, выбор модели.

Когда что:

Сценарий	Что использовать
Я разрабатываю проект, направляя Claude	Claude Code
Веб-приложение шлёт один промпт пользователю	Claude API
Скрипт-демон ловит данные, использует tools, шлёт результат	Agent SDK
CI/CD: автоматический code review на каждом PR	Agent SDK

Agent loop: внутренний цикл агента в коде

Когда вы пишете промпт в Claude Code и Claude вызывает tools – цикл крутится *внутри* Claude Code, вы его не видите. В Agent SDK этот же цикл – **ваш собственный код**:

```
messages = [{"role": "user", "content": "проанализируй акцию AAPL"}]

while True:
    response = client.messages.create(
        model="claude-sonnet-4-6",
        tools=my_tools,          # список tools, которыми может пользоваться
        messages=messages,
    )
    messages.append({"role": "assistant", "content": response.content})

    if response.stop_reason == "end_turn":
        # модель закончила
        break

    if response.stop_reason == "tool_use":
        # модель просит вызвать tool
        for tool_call in response.content:
            if tool_call.type == "tool_use":
                result = execute_tool(tool_call.name, tool_call.input)
                messages.append({"role": "user", "content": [
                    {"type": "tool_result", "tool_use_id": tool_call.id, "c
                ]})
            # цикл повторяется: модель видит tool_result и продолжает
```

Это и есть «agent loop». Claude Code прячет его, SDK — даёт в руки.

Вы контролируете каждую итерацию: можете остановить раньше, поменять tools на ходу, добавить логи, тайм-ауты, защиту.

Skill `claude-api` делает шаблон ещё лучше: добавляет prompt caching, retry на ошибках, ограничение по шагам цикла, защиту от «infinite tool call».

Headless mode и CI/CD: Claude как «функция»

Headless mode — запуск Claude в окружении без интерактивного терминала. Сервер, CI-pipeline, daemon — никто не ждёт ваших ответов на вопросы Claude.

Что это меняет:

- Никаких «*Claude спросил подтверждение, я нажал у*». Все разрешения — в коде заранее.
- Все секреты — через env vars, не через интерактивный prompt.

- Тайм-ауты обязательны. Зависший агент в CI – падает билд.
- Логирование – обязательное и структурированное (JSON), чтобы потом разобрать.

Пример – GitHub Action для auto code review:

```
# .github/workflows/claude-review.yml
name: Claude Code Review
on:
  pull_request:
    types: [opened, synchronize]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install anthropic
      - run: python .github/scripts/claude_review.py
    env:
      ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
      GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

Скрипт `claude_review.py` через Agent SDK:

1. Получает diff PR через GitHub API;
2. Шлёт в Claude с промптом «найди 3–5 проблем»;
3. Через tool `post_comment` – постит замечания прямо в PR.

Tool definitions для SDK: не MCP, а Python-функции

В Claude Code мы давали Claude tools *через MCP*. В SDK – через определения прямо в коде:

```
my_tools = [
    {
        "name": "get_price",
        "description": "Возвращает текущую цену акции по тикеру.",
        "input_schema": {
            "type": "object",
            "properties": {
                "ticker": {"type": "string", "description": "тикер, e.g. AAPL"}
            },
            "required": ["ticker"]
        }
    },
    {
        "name": "post_telegram",
        "description": "Шлёт сообщение пользователю в Telegram.",
        "input_schema": {
            "type": "object",
            "properties": {
                "text": {"type": "string"}
            },
            "required": ["text"]
        }
    }
]
```

Когда Claude решает вызвать tool, ваш цикл получает `tool_use` с именем и аргументами — и вы *сами* вызываете соответствующую Python-функцию. То есть SDK не интегрируется с MCP — tools определяются и исполняются у вас в коде.

Зачем разница: в CI или daemon — нет процесса для MCP-сервера, нет конфига Claude Code. SDK даёт самодостаточный, переносимый агент в одном файле.

Prompt caching и batch API: оптимизация стоимости

Prompt caching — вы уже видели в этапе 06. В SDK активируется одним полем:

```
response = client.messages.create(  
    model="claude-sonnet-4-6",  
    system=[  
        {"type": "text", "text": "Большой системный промпт..."},  
        {"cache_control": {"type": "ephemeral"}}  
    ],  
    messages=messages,  
)
```

Эта часть кэшируется на 5 минут. На повторных вызовах с тем же системным промптом стоимость падает в 10 раз. Идеально для Twitter-демона, где каждый твит идёт через один и тот же «инструктаж».

Batch API — асинхронный режим: отправляете 100 промптов одной пачкой, ответы готовы через ~30 минут, цена в 2 раза ниже. Подходит для «не срочных» задач: ночная классификация всех вчерашних твитов, переоценка портфеля раз в день, обработка длинной очереди задач.

Не подходит — для интерактивных бот-задач, где пользователь ждёт ответ.

Безопасность headless: что отличается от Claude Code

В Claude Code вы нажимаете «yes/no» на каждое опасное действие.

В headless такого нет — всё решается заранее. Что обязательно:

1. **Список разрешённых tools — узкий.** В Twitter-демоне нет смысла давать tool «удалить файл». Ограничьте только теми, что нужны.
2. **Ограничение шагов цикла.** `max_iterations = 10`. Иначе при ошибке агент будет крутиться вечно, жгя токены.
3. **Тайм-аут на запрос.** 60 секунд — и убиваем. В CI это критично: иначе билд висит.
4. **Аудит-лог.** Каждое действие — в лог-файл с timestamp. PostHog (этап 10) для агрегации.
5. **Бюджет.** На запуск и на день. Если превысили — останавливаемся, шлём алерт.
6. **Sandboxing.** Если агент пишет в файлы — пусть пишет в tmp/, а не в корень репо. В CI — в artifact-папку.

В Claude Code эти правила «по умолчанию». В SDK — ваша ответственность.

Долговременная память агента: разговор «с продолжением»

Агент в скрипте по умолчанию забывает всё после `break`. Для Telegram-консьержа нужно «помнить вас»: что любите, что искали вчера, какие задачи стоят.

Шаблоны:

1. **В БД** – `conversations(user_id, messages_json)`. Загружаете при старте сессии, сохраняете перед выходом.
2. **В файле на диск** – `~/.concierge/<user_id>.json`. Простое, для одного пользователя.
3. **Через summarisation** – когда история длинная, прогоняете её через Claude и получаете «сжатую память». Дальше работаете с summary, а не с полной историей. Экономит токены.

Дисциплина:

- Не храните пароли и ключи в истории сообщений – это утечка ждущая случиться;
 - Шифруйте, если хранится в облаке (этап 10, дневник на автопилоте);
 - Дайте пользователю команду `/forget` – удалить всю историю одной кнопкой. GDPR.
-

PRACTICUM

Прогон · агент с одним tool за 15 минут

1. Создайте папку `hello-agent`, в `.env` — `ANTHROPIC_API_KEY`.

2. Промпт:

```
> используя skill claude-api,  
> составь Python-скрипт agent.py:  
> - один tool «get_time», возвращает текущее время в ISO;  
> - agent loop с max_iterations=5;  
> - prompt: «который сейчас час и какой день?»;  
> - модель claude-haiku-4-5 (дешёвая для теста).  
> запусти, покажи трассу: какие tool_use появились, что Claude ответил.
```

3. Проверьте, что Claude действительно вызвал `get_time` через tool, а не «придумал» из своих знаний.

4. Усложните: добавьте второй tool «post_telegram» (если Telegram-бот из этапа 10 готов). Промпт:

```
> модифицируй agent.py: добавь tool post_telegram(text).  
> новый prompt: «получи время и пришли мне в telegram  
> «доброе утро» с этим временем».  
> убедись, что агент сделал ДВА tool_use по очереди:  
> сначала get_time, потом post_telegram, и завершился.
```

5. Проверка идиомпотентности: запустите дважды — должны прийти два сообщения.

6. Бюджет токенов: добавьте `max_tokens=200` на запрос. Запустите ещё раз. Логи — через PostHog skill этапа 10.

Что мы тренируем: **скелет агента в 5–10 строк Python**, который сам решает, когда вызывать tools, и завершается, когда задача выполнена.

ПРОЕКТ А · ПО ИНСТРУКЦИИ

Twitter-сигналы в реальном времени

Цель: Python-демон на Railway, ловит твиты по вашим тикерам через X API. Через Claude Agent SDK – sentiment + влияние на цену. При срабатывании порога – push в Telegram. Идея «trading signals», но используем для обучения, не для реальных сделок.

Шаги

1. Создайте репо `twitter-signals/`. Структура:

```
twitter-signals/
├── daemon.py          # главный цикл
├── agent.py           # Claude Agent SDK
├── tools/
│   ├── price.py      # get_price через Alpha Vantage
│   └── telegram.py    # post_telegram
├── data/
│   └── tickers.json   # тикеры для слежки
├── .env.example
└── railway.json
```

2. Если у вас есть X API – `daemon.py` подключается к streaming-эндпоинту и ловит твиты. Если нет – demo-режим, читает `data/sample-tweets.json` по одному в 5 секунд.

3. `agent.py` – Claude Agent SDK с тремя tools:

- `get_price(ticker)` – текущая цена;
- `get_my_position(ticker)` – есть ли у меня эта акция (читает `portfolio-journal/data/operations.csv`);
- `post_telegram(text)` – уведомление.

4. Системный промпт (с prompt caching, потому что одинаков на каждом твите):

Ты — финансовый аналитик-скептик. Каждый твит про акцию оцени по шкале 1–10:

- sentiment (1 = очень плохо, 10 = очень хорошо);
- вероятность влияния на цену в ближайший день;
- твоя степень уверенности.

Если sentiment < 3 ИЛИ > 8 И влияние > 7 И уверенность > 6 —
И у пользователя есть позиция (через get_my_position) —
ОДИН раз вызови post_telegram с кратким резюме.

Иначе — ничего не делай, просто заверши.

Никаких советов «купить/продать». Только наблюдение.

5. Дисциплина (см. § I):

- `max_iterations = 5` в agent loop;
- `max_tokens = 300` — короткие ответы;
- модель **haiku** — дёшево для потока твитов;
- дневной бюджет \$0.50 на агента, прерываемся при превышении;
- идемпотентность: `data/sent-alerts.json` — не шлём один и тот же сигнал дважды.

6. Деплой на Railway как «worker»-сервис (не cron, а всегда работающий процесс):

```
{
  "deploy": {
    "startCommand": "python daemon.py",
    "restartPolicyType": "ON_FAILURE",
    "numReplicas": 1
  }
}
```

7. Verification:

```
> запусти red-team subagent. brief:  
> представь, что я случайно скачал tweet-flood  
> (100 твитов в минуту). что произойдёт с моим бюджетом  
> и API ключом? какие три защиты мне нужны?
```

8. Дисклеймер обязателен: каждый push в Telegram заканчивается строкой
«наблюдение, не инвестиционный совет».

AI-консьерж в Telegram

Цель: Telegram-бот на Agent SDK с долговременной памятью. Любые задачи: «куда поехать на 3 дня в апреле, бюджет 50к», «найди подарок для сестры, любит керамику», «меню на неделю под мои предпочтения». Бот помнит ваши прошлые запросы и подстраивается.

Шаги

1. Репо `concierge-bot/`. На Railway worker-сервис.
2. Память: PostgreSQL-таблица `memory(user_id, key, value, updated_at)`.
Хранит «факты» о пользователе: предпочтения, города, бюджет по умолчанию, история запросов.
3. Tools для агента:
 - `remember(key, value)` – запомнить факт;
 - `recall(key)` – извлечь факт;
 - `web_search(query)` – поиск в интернете;
 - `web_fetch(url)` – чтение страницы;
 - `send_message(text)` – ответ пользователю.

4. Системный промпт:

Ты – личный ассистент. Перед каждой задачей:

- 1) `recall` интересные факты про пользователя (предпочтения, бюджеты);
- 2) если задача требует свежих данных – `web_search` и `web_fetch`;
- 3) сформулируй конкретный ответ: 3-5 пунктов, не «возможно»;
- 4) если узнал что-то новое о пользователе – `remember`.

НЕ выдумывай факты. Если не знаешь – спрашивай через `send_message`.

5. Цикл: получили сообщение от пользователя → запустили agent loop с историей последних 10 сообщений + всеми `recall` -ами → агент работает → завершился → сохраняем последний ответ в `memory`.

6. Команды бота:

- `/forget` – очистить всю память (GDPR);
- `/recall` – показать, что бот про вас «знает»;
- `/budget` – показать сколько токенов потрачено за месяц.

7. Test-drive:

```
[вы]    привет, я люблю керамику и небольшие путешествия
[бот]   записал 🙌 чем могу помочь?
[вы]    куда поехать на 3 дня в апреле, бюджет 50к
[бот]   с учётом «небольшие путешествия» и «керамика»:
        1) Гжельский завод + Подмосковье...
        2) Сетка Псков + псковская керамика...
        3) Фарфор Императорский завод (Питер)...
[вы]    /recall
[бот]   про вас:
        - любит керамику
        - предпочитает небольшие путешествия
        - типичный бюджет на 3-дневную поездку: ~50к
```

8. Безопасность: `max_tokens=1500`, `max_iterations=8`, дневной бюджет на пользователя \$1, шифрование `memory.value` в БД (как в дневнике этапа 10).

9. Verification: запустите subagent с ролью «privacy-аудит». Brief: представь, что бот утёк через дамп БД. Что увидит атакующий? Что нельзя восстановить?

Свои агенты в вашей инфраструктуре

ПРОЕКТ С · САМОСТОЯТЕЛЬНО

GitHub Action: автоматический Claude code review на каждый PR

Цель: вариация проекта А – тот же стек (*headless agent*), но другая среда (CI). При каждом PR в ваш репо `my-personal-office` (этап 09) или `my-investing` (этап 06) GitHub Action запускает Claude Agent SDK – и он через tool `post_review_comment` постит замечания прямо в PR.

Подсказки

- В `.github/workflows/claude-review.yml` – триггер `pull_request: opened, synchronize`.
- В `.github/scripts/claude_review.py` – ваш агент. Получает diff через `gh pr diff`, постит комментарии через `gh pr review --comment`.
- Tools агента: `get_diff`, `read_file`, `post_review_comment`, `finalize_review(verdict)`.
- Системный промпт: ищем 3–5 проблем, никаких style-наг (это для линтеров), фокус на баги/безопасность/неочевидное.
- Бюджет: `max_tokens=2000`, `max_iterations=10`, тайм-аут 5 минут (CI должно быстро отвечать).
- Поставьте на один из ваших проектов – и откройте тестовый PR.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

– сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ в вашем репо есть `.github/workflows/claude-review.yml` и `scripts/claude_review.py`;
- ✓ при открытии тестового PR через 1–2 минуты появляются комментарии Claude;
- ✓ замечания осмысленные (не «увеличьте отступ», а «здесь возможен SQL-инъекция»);
- ✓ тайм-аут срабатывает (тестировали с `max_iterations=1` – агент не зависает);
- ✓ через PostHog видно стоимость каждого review;

✓ агент не имеет права делать ничего, кроме `post_review_comment` — ВЫ ЯВНО ограничили.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Запусти *subagent* с ролью «*security-инженер*». *Brief*: посмотри мой *workflow* и *script*.
Найди три места, где злонамеренный *PR* (от человека, который форкнул мой репо) может через мой *workflow* исчерпать мой *ANTHROPIC_API_KEY* или утечь его. Как защититься?»

«Утренний агент» — кастом под вашу повседневность

Цель: вариация проекта B (агент с долговременной памятью), но для не универсальных задач — для вашего конкретного утреннего ритуала. Например: «каждое утро в 7:30 агент проверяет мой Google Calendar, погоду, выбирает оптимальный маршрут до первой встречи с учётом пробок, шлёт one-message сводку».

Подсказки

- Это та же архитектура, что в B (Agent SDK + tools + долговременная память), но другие tools — под ваш ритуал.
- Запуск — cron из этапа 10 (не daemon, потому что один раз в день).
- Возможные tools: Google Calendar API, Open-Meteo (погода без ключа), 2GIS/Google Maps (для маршрута), Telegram для ответа.
- Долговременная память: ваши «правила» (любите кофе перед встречами, не ходите в спортзал по четвергам), которые агент учитывает в сводке.
- Длина: ровно один Telegram-message, читается за 30 секунд за утренним кофе.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

— сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ агент стартует по cron в 7:30 каждое утро без вашего участия;
- ✓ в Telegram приходит ровно одно сообщение на 30 секунд чтения;
- ✓ сообщение учитывает ваши «правила» из памяти (тестировали — добавили правило, на следующее утро оно отразилось);
- ✓ есть `/forget` и `/recall` — как в консьерже;
- ✓ стоимость одного запуска видна в PostHog (по опыту — 3–5 центов);
- ✓ в течение недели вы ни разу не трогали агента руками — работает сам.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Прогуляйся по неделе моих сводок как «коуч продуктивности». Brief: что в моих утренних сообщениях повторяется (и потому шумит), что важно но иногда теряется? Какую структуру я мог бы попробовать на следующей неделе?»

Claude Agent SDK

Библиотека Anthropic (anthropic для Python, @anthropic-ai/sdk для Node), позволяющая запускать Claude как агента из вашего скрипта. Полный agent loop, tools, prompt caching – всё в ваших руках.

Agent loop

Внутренний цикл агента: *prompt* → *tool calls* → *tool results* → *next prompt* → ... до `stop_reason: end_turn`. В Claude Code скрыт, в SDK – ваш код.

Headless mode

Запуск Claude без интерактивного TTY: на сервере, в CI, в daemon-скрипте. Все разрешения, секреты, тайм-ауты – в коде заранее.

Tool definition

Описание tool в SDK: имя, description, JSON-schema параметров. Передаётся в `messages.create(tools=...)`. Когда модель решает вызвать tool – ваш код исполняет соответствующую функцию.

Daemon

Долгоживущий процесс, постоянно работающий в фоне. На Railway – «worker»-сервис. Используется для стримов (Twitter), ботов (Telegram), мониторов.

Batch API

Асинхронный режим Anthropic API: 100 промптов одной пачкой, ответы через ~30 минут, в 2 раза дешевле. Подходит для несрочных задач (ночная классификация, переоценка портфеля).

CI/CD

Continuous Integration / Continuous Deployment – автоматизация сборки, тестов и деплоя. В курсе – через GitHub Actions: на каждый push/PR запускается набор скриптов, в т. ч. Claude code review.

GitHub Actions

Встроенная CI-система GitHub. Workflow описывается YAML-файлом в `.github/workflows/`. На триггер – запускает шаги: setup среды, выполнение скриптов, постинг результатов.

max_iterations

Ограничение количества шагов agent loop. Без него агент при ошибке может крутиться вечно. Стандартно – 5–10 для большинства задач.

Долговременная

Хранение информации о пользователе между сессиями агента. БД,

память

файл, или сжатая через summarisation. Обязательная команда
/forget для GDPR.

01 В каком сценарии лучше Claude API, а не Agent SDK?

Когда вам нужно *один* ответ на один промпт, без tools и без цикла. Например: чат-приложение, которое получает вопрос и возвращает ответ. Claude API проще, дешевле и не требует определять tools.

02 Что делает `stop_reason: tool_use` в agent loop?

Это сигнал: «модель просит тебя вызвать tool». Ваш код должен исполнить запрошенные tool-функции, передать результаты обратно как `tool_result`, и продолжить цикл – модель увидит результаты и либо вызовет ещё tool, либо завершит с `end_turn`.

03 Зачем обязательно ограничение `max_iterations` в headless?

При ошибке (плохой tool, неудачный prompt, цикл из «попробуй снова») агент может крутиться вечно, жгя токены. В Claude Code пользователь нажмёт Esc – в headless этого нет. `max_iterations` – страховка: после N шагов – принудительный выход с ошибкой, которую можно увидеть в логах.

04 Где живут tools в Agent SDK – в MCP-сервере или в вашем коде?

В вашем коде. SDK – это *агент в скрипте*, без Claude Code-обёртки и без конфигов MCP. Tools определяются как Python-функции (или Node) и передаются в `messages.create(tools=...)`. Это даёт переносимость – один файл-скрипт работает в CI, на сервере, в любом окружении.

05 Когда уместен **Batch API**, а когда нет?

Уместен: большие пачки несрочных задач – ночная классификация, переоценка портфеля раз в день, обработка очереди задач. В 2 раза дешевле, ответ через ~30 минут. **Не уместен:** интерактивные сценарии, где пользователь ждёт ответ (Telegram-бот, AI-консьерж).

06 Зачем команда `/forget` в AI-консьерже?

GDPR и здравый смысл. Бот хранит факты о пользователе – пользователь должен иметь возможность их удалить одной кнопкой. Без `/forget` ваш бот не соответствует европейскому регулированию (а кроме того – неэтичен).

07 Какие шесть мер безопасности обязательны в **headless**-агенте?

1) Узкий список разрешённых tools; 2) `max_iterations` на agent loop; 3) тайм-аут на каждый запрос; 4) аудит-лог в файл/PostHog; 5) бюджет (на запуск, на день); 6) sandboxing файловых операций (tmp/ или artifact-папка). В Claude Code это «по умолчанию», в SDK – ваша ответственность.

Что я теперь умею

- ☐ Различаю Claude Code, Claude API и Claude Agent SDK – и знаю, когда какой нужен.
 - ☐ Понимаю agent loop и могу прочитать/написать его в 5–10 строк Python.
 - ☐ Определяю tools для SDK через JSON-schema и связываю с Python-функциями.
 - ☐ Запускаю Claude в headless: на daemon-сервере и в GitHub Actions.
 - ☐ Применяю шесть мер безопасности headless (tools, max_iterations, тайм-аут, лог, бюджет, sandbox).
 - ☐ Делаю долговременную память агента с командой `/forget` и шифрованием.
 - ☐ Сделал четыре проекта: Twitter-сигналы, AI-консьерж, GitHub-action review, утренний агент.
-
-

Готовы закрыть этап? Нажатие фиксирует прогресс на главной странице.

«*Claude Code* — это для диалога. *Claude Agent SDK* — для того, что должно работать без вас. Граница между ними — в том, есть ли у *Claude* собеседник».

ВНУТРИ ЭТАПА

- § 0 · Подготовка
- § I · Темы
- § III · Проекты
- § V · Словарь
- § VI · Quiz
- § VII · Чек-лист

КУРС

- К оглавлению
- ← Этап 10
- Финальный капстоун